

Chapter 3

Indexing

Finding a row in $\log B$ page reads instead of B , and knowing when not to

Advanced Data Management

Based on Chapter 3 lecture notes and Lab 3

How to read this chapter

Chapter 2 ended on a painful number: finding one row in a heap of B pages costs B page reads, about 5,000 on average for a million-row table. This chapter is about the data structure that turns that into 3 or 4. But the second half of the chapter is just as important as the first: an index is not free, and the optimiser will sometimes ignore one you created. By the end you should be able to draw a B+ tree, count the page reads for a lookup and a range, decide which index type fits a query, and predict when the planner will refuse to use it. Lab 3 measures all of this; Section 8 walks through it.

1 Refresher: Cost, Trees, Hashing

1.1 How growth rates compare

We measure cost in pages touched. The *shape* of that cost as the table grows matters enormously.

Rows N	$O(N)$ scan	$O(\log_2 N)$	$O(1)$
1,000	1,000	about 10	1
1,000,000	1,000,000	about 20	1
1,000,000,000	10^9	about 30	1

Read the middle column slowly. The table grew a million-fold from the first row to the last, and the logarithm went from 10 to 30. That is the whole point of indexing: move a lookup from the first column to the second or third, where growth barely registers.

Key insight. $\log N$ is the closest thing to a free lunch in computer science. Doubling the data adds *one* step.

1.2 Balanced search trees, and why fanout matters

A binary search tree finds a key in $O(\text{height})$ steps: at each node, go left or right. The catch is that height depends on insertion order. Insert keys in sorted order and the tree degenerates into a linked list of height N , and search is back to $O(N)$. **Balanced** trees (AVL, red-black) fix this by rebalancing on insert so that height stays about $\log_2 N$ regardless of order.

Databases go one step further. A binary tree has two children per node, so a million keys need about 20 levels, and each level is a page read. Twenty page reads is better than five thousand but still slow. The fix is **fanout**: let each node have hundreds of children instead of two. Then the height is $\log_f N$ for fanout f , and with $f = 100$ a million keys sit only three levels deep, a billion keys only four or five.

Worked example: Where does fanout 100 come from?

A node is one 8 KB page. If a routing key is an 8-byte integer and a child pointer is 8 bytes, each (key, pointer) pair costs 16 bytes, so a page holds about $8192/16 \approx 500$ of them. Real fanouts are lower because of page headers, larger keys (strings), and pages being on average only about two-thirds full, so “about 100” is a conservative, round estimate. Even so:

$$\log_{100} 10^6 = 3, \quad \log_{100} 10^9 = 4.5 \Rightarrow 5.$$

Why is fanout so effective? Because we pay per *page*, not per comparison. Comparing a key against 100 routing keys inside one page is cheap CPU work; what costs is fetching the page. A high-fanout tree spends the page’s 8 KB on as many routing decisions as possible, so each page read eliminates 99% of the remaining candidates rather than 50%.

1.3 Hashing

A hash function maps a key to a bucket: $h(\text{key}) \rightarrow \text{bucket number}$. Average lookup is $O(1)$: compute the hash, go straight to that bucket. Collisions (two keys, one bucket) are handled by chaining (a list per bucket) or probing (try the next bucket). The price is that hashing **destroys order**: keys 41 and 42 land in unrelated buckets, so “all keys between 40 and 70” means looking everywhere.

Concept: The contrast to keep in mind

Trees preserve order, so they support equality *and* ranges. Hashing does not, so it supports equality only. Everything in this chapter about choosing an index type comes back to this.

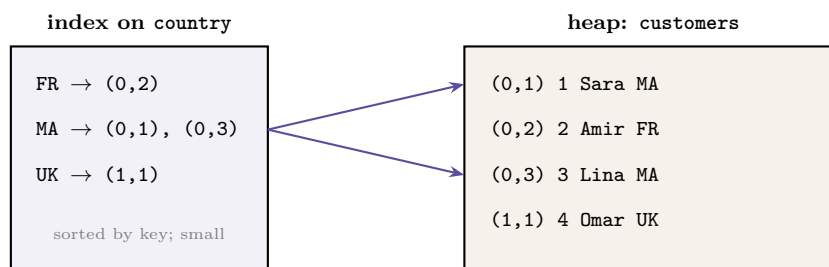
2 Why Indexes?

2.1 What an index is

Concept: Index

An index is a **separate data structure** that maps key values to the locations of matching rows. The table (the heap) stays exactly as it is; the index sits beside it and points in.

The book analogy is exact. A textbook’s pages are the heap: content in whatever order it was written. The index at the back is a separate structure, sorted by term, where each entry says “page 212”. To find “fanout” you do not read the book; you look up the term and jump. And note two things the analogy also gets right: the index takes up pages of its own, and every time the author adds a section, the index has to be updated too.



The locations the index stores are the (page, slot) row ids from Chapter 2, PostgreSQL’s `ctid`. That is why the slotted page’s stable ids mattered: an index can point at a row and stay correct even when the row moves within its page.

Warning / Gotcha: The trade-off, stated up front

An index makes matching *reads* fast. In exchange, every *write* to the table must also update the index, and the index occupies space. Indexing is a deliberate choice, not a free win. The rest of the chapter keeps returning to this.

2.2 Selectivity: the deciding factor

Concept: Selectivity

The **selectivity** of a predicate is the fraction of the table’s rows it returns. `WHERE id = 4275` returns one row in a million: selectivity 10^{-6} , “highly selective”. `WHERE amount > 0` returns almost everything: selectivity near 1, “not selective”.

(The notes say “low selectivity” for few rows matching. Vocabulary varies between textbooks; what matters is the fraction, so I will say “few rows” and “most rows” to avoid the ambiguity.)

Here is the counter-intuitive fact that this whole section builds toward: **having an index does not mean the optimiser will use it**. To see why, count pages.

Worked example: When the index loses: counting the reads

The `orders` table has $B = 12,821$ pages holding 1,000,000 rows (78 per page). There is a B+ tree on `amount`, height 3. Compare two predicates. **WHERE `amount` = 57.25**, matching about 10 rows. Index scan: 3 reads to reach the leaf, then one heap page read per matching row, at most 10, so about 13 page reads. Sequential scan: 12,821. The index wins by a factor of a thousand.

WHERE `amount` > 20, matching about 85% of rows, 850,000 of them. Index scan: 3 reads to reach the first leaf, then walk the leaves, and for *each* matching row fetch its heap page. Those rows are scattered across the heap in `amount` order, which has nothing to do with heap order, so consecutive index entries point at unrelated pages: roughly one *random* page read per row. That is up to 850,000 page reads, many of them re-reading the same 12,821 pages over and over. Sequential scan: 12,821 reads, in order, streaming from disk. The sequential scan wins by a factor of sixty, even though it reads every row and “wastes” 15% of them.

So the rule: if a query returns a small fraction of the table, the index is a huge win; if it returns most of the table, scanning sequentially beats jumping around. The optimiser estimates the fraction from catalog statistics and picks accordingly. Lab 3, Part 4, has you find the exact crossover experimentally by varying the threshold on `amount` and watching the plan flip from Index Scan to Seq Scan.

Key insight. An index scan costs roughly one *random* page read per matching row. A sequential scan costs B *sequential* reads regardless. The crossover is usually at a few percent of the table.

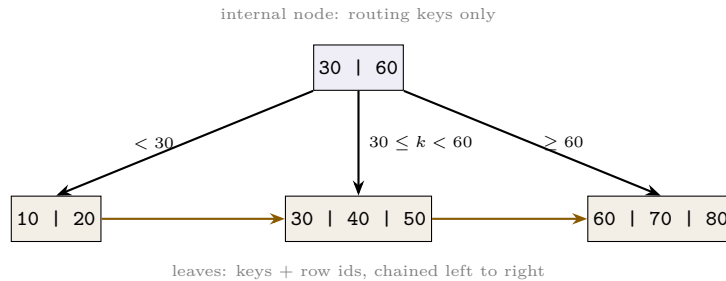
Summary

An index is a separate, sorted structure mapping keys to row ids, beside the heap. Reads on the key get faster; every write gets slower and space is consumed. Whether the optimiser uses an index depends on selectivity: roughly one random page read per matching row for an index scan, versus B sequential reads for a scan. Few rows: index. Most rows: scan.

3 B+ Trees

3.1 Conceptual overview: the structure

The B+ tree is the default index in essentially every relational database, and once you see why, the choice is obvious. It is a balanced, high-fanout search tree with one extra design decision: *all the data lives in the leaves*, and the leaves are chained together left to right.



Mechanism: B+ tree

- **Internal nodes** hold routing keys and child pointers, nothing else. A node with keys $k_1 < k_2 < \dots$ has one more pointer than keys: the child for keys below k_1 , the child for keys in $[k_1, k_2)$, and so on.
- **Leaves** hold the actual index entries: (key, row id) pairs, sorted by key. Every key in the table appears in exactly one leaf.
- **Leaves are linked** in a chain, left to right, so that once you are at a leaf you can walk forward to the next keys without going back up the tree.
- Every leaf is at the same depth: the tree is perfectly balanced, always.
- Each node is one page, so every node visited is one page read.

Two questions students always ask. *Why keep the data only in the leaves?* Because it lets internal nodes hold nothing but routing keys, which maximises fanout; and it makes every search end at the same level, which makes range scans possible. *Why the plus?* The original B-tree stores data in internal nodes too. The B+ variant, with data only in leaves and a leaf chain, is what databases actually use, and people say “B-tree” loosely to mean it. PostgreSQL’s “btree” index type is a B+ tree.

3.2 Searching

Worked example: Tracing a search for key 40

1. **Read the root** (page read 1). Routing keys are 30 and 60. Is $40 < 30$? No. Is $40 < 60$? Yes. So follow the middle pointer.
 2. **Read the middle leaf** (page read 2). Keys 30, 40, 50. Scan it (in memory, cheap) and find 40 with its row id, say (417, 12).
 3. **Read heap page 417** (page read 3) and pull slot 12: the row.
- Three page reads. For a tree of height h , a point lookup costs h reads to reach the leaf plus one to fetch the row: $h + 1$. In practice the root and often the whole second level stay in the buffer pool (Chapter 6), so the real disk cost is often one or two reads.

With fanout $f \approx 100$ and $N = 1,000,000$ rows, $h = \lceil \log_{100} 10^6 \rceil = 3$, so any row is found in about four page reads versus thousands for a heap scan. If the key is absent, the search still ends at the leaf where it *would* be, sees it is not there, and stops: h reads. Compare the heap, which had to read all B pages

to confirm absence.

3.3 Range scans: the chained leaves

Worked example: WHERE amount BETWEEN 40 AND 70

1. Descend once, exactly as for a point lookup, to the leaf holding the first match, 40. (h reads.)
 2. Walk rightward along the leaf chain: 40, 50 in this leaf; follow the link to the next leaf: 60, 70; then 80, which is past 70, so stop. (One read per leaf visited.)
 3. For each matching entry, fetch its row from the heap. (One read per row, possibly fewer if several rows share a page.)
- Cost: $h + (\text{leaves spanned}) + (\text{rows fetched})$. The tree is descended *once*, not once per key.

This is why the leaf chain exists, and it is exactly what a hash index cannot do. Sorting is also served this way: **ORDER BY amount** can be answered by walking the leaves from the start, with no sort step at all.

3.4 Inserts and staying balanced

The sorted file of Chapter 2 had $\log_2 B$ search but catastrophic inserts, because keeping a flat file in order can shift everything. The B+ tree keeps the search cost and fixes the insert, by never shifting more than a page or two.

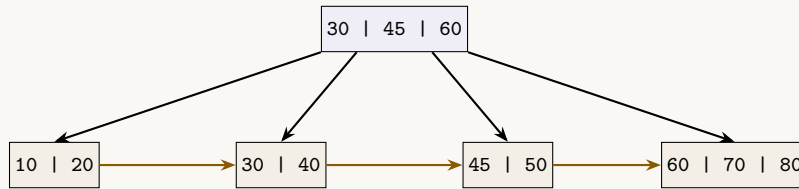
Mechanism: Insert

1. Search for the key's position; this lands on the correct leaf.
2. If the leaf has room, insert the entry in sorted position. Done: cost h reads and 1 write.
3. If the leaf is **full**, **split** it: allocate a new leaf, move the upper half of the entries into it, link it into the chain, and push the first key of the new leaf up into the parent as a new routing key.
4. If the parent is now full, split it too, pushing a key further up. A split can cascade all the way to the root; when the root splits, a new root is created and the tree grows one level taller.

Worked example: Tracing a split

Leaves hold at most 3 entries. Insert 45 into the tree drawn above.

1. Search: $45 < 60$ and ≥ 30 , so the middle leaf (30, 40, 50). It is full.
2. Split: keep (30, 40) in the old leaf, move (45, 50) into a new leaf, and chain them: $\dots \rightarrow (30, 40) \rightarrow (45, 50) \rightarrow (60, 70, 80)$. The separator pushed up is 45, the first key of the new leaf.
3. The root becomes (30 | 45 | 60), with four children. It had room, so no cascade.



Only two leaves and one internal node were touched. The other 10,000 leaves of a real tree are untouched. That locality is why B+ tree inserts cost $O(h)$ and not $O(B)$.

Deletes are symmetric: remove the entry, and if a leaf becomes less than half full, borrow from a sibling or merge with it. Real systems (PostgreSQL included) often skip the merge and simply let underfull pages sit until a later reorganisation, because merges are expensive and inserts usually refill them.

Trade-offs: B+ tree versus the Chapter 2 file organisations

Operation	Heap	Sorted file	Hashed	B+ tree
Equality search	B	$\log_2 B$	~ 1	$h + 1 \approx 4$
Range search	B	$\log_2 B + \text{matches}$	B	$h + \text{leaves} + \text{matches}$
Insert	~ 1	up to B	~ 1	$h + 1$, occasionally a split

The B+ tree gets the sorted file's read behaviour with inserts that cost a handful of pages instead of the whole file. That is why it is the default everywhere.

3.5 Clustered versus secondary indexes

So far the index points into a heap whose rows are in arbitrary order. There is one more choice.

Trade-offs: Clustered and secondary

Clustered

The table's rows are physically stored in index order.

A range scan reads contiguous heap pages: very fast, since matching rows sit next to each other.

Only one per table: the data can be in only one physical order.

Secondary

Index order differs from table order.

Leaves point into the heap by row id; matching rows are scattered, so a range means many random reads.

As many as you like per table.

Return to the `amount > 20` example. With a *clustered* index on `amount`, the 850,000 matching rows would occupy contiguous heap pages, and the index

scan would cost about $3 + 0.85 \times 12,821 \approx 10,900$ sequential reads, competitive with the full scan. It was the *scattering* of a secondary index that made it lose. In PostgreSQL every index is secondary by default; the `CLUSTER` command physically reorders a table by a chosen index, once, and new inserts are not kept in that order. MySQL’s InnoDB, by contrast, always clusters the table by its primary key.

Summary

A B+ tree keeps routing keys in internal nodes and all entries in chained, sorted leaves; every node is a page and the tree is always balanced. Point lookup: $h + 1$ reads, about 4 for a million rows. Range: descend once, walk the leaves. Insert: find the leaf, split if full, push a separator up; cost $O(h)$. Clustered means the heap is in index order (one per table, great ranges); secondary means it is not (many per table, scattered ranges).

4 Beyond B-Trees

The B+ tree is the default, but several other index shapes fit specific patterns better. For each one, the question to ask is: *what pattern of queries is this built for, and what does it give up?*

4.1 Hash indexes

Map the key through a hash function to a bucket: $O(1)$ equality lookup, one or two page reads regardless of table size. Order is lost, so no ranges, no `ORDER BY`, no prefix matches on strings. Worth it only when a column is used *exclusively* for exact-match lookups, and even then the gain over a B+ tree of height 3 is small. In PostgreSQL, `CREATE INDEX ... USING hash`.

4.2 Bitmap indexes

For **low-cardinality** columns, those with few distinct values such as `country` (5 values) or `status` (3 values), a B+ tree is a poor fit: each key has hundreds of thousands of entries. A bitmap index instead keeps *one bitmap per value*: bit i of the `MA` bitmap is 1 if row i has `country = 'MA'`.

row	1	2	3	4	5	6	7	8
<code>country = MA</code>	1	0	1	0	0	1	1	0
<code>country = FR</code>	0	1	0	0	1	0	0	0
<code>status = active</code>	1	1	0	1	0	1	0	1
<code>MA AND active</code>	1	0	0	0	0	1	0	0

The power is in combining predicates. The condition “country is MA and status is active” is a bitwise AND of two bitmaps, which a CPU does 64 rows at a time. A million-row bitmap is only 125 KB. Bitmaps are common in

analytics and data warehouses; they are poor for frequently updated tables, because every update touches a bit in every bitmap of that column.

4.3 Partial, expression and covering indexes

These three tailor an index to a known query pattern.

- **Partial index:** index only the rows satisfying a condition. `CREATE INDEX ... ON orders(id) WHERE status = 'active'`. If 2% of orders are active and every query asks for active ones, the index is 2% of the size, fits in memory, and is cheaper to maintain, because inserts of inactive orders do not touch it at all.
- **Expression index:** index a function of a column rather than the column itself. An index on `lower(name)` serves the predicate `lower(name) = 'sara'`. A plain index on `name` cannot, because the tree is sorted by `name` and the query asks about a different value. Lab 3, Part 5 demonstrates exactly this failure and fix.
- **Covering index, giving an index-only scan:** include in the index every column the query needs, so the heap is never touched. An index on `cust_id` that also stores `amount` lets “sum of amount for customer 42” be answered from the leaves alone. That removes the “one heap read per match” term entirely.

In the real world.

PostgreSQL does not store bitmap indexes. Instead it builds a bitmap *on the fly* from an ordinary B+ tree: the “Bitmap Index Scan” collects matching row ids into a bitmap, and the “Bitmap Heap Scan” then reads the heap pages in physical order, each page once. This is its answer to the scattered-reads problem of the previous section.

4.4 Multi-column indexes and the leftmost-prefix rule

An index on `(country, city)` is a B+ tree sorted first by `country`, then by `city` within each country, like a phone book sorted by surname then first name. That ordering determines which queries it can serve.

Predicate	Uses it?	Why
<code>country = 'MA'</code>	yes	the leading column; all MA entries are contiguous
<code>country = 'MA' AND city = 'Rabat'</code>	yes	narrows further within the MA run
<code>city = 'Rabat' alone</code>	no	Rabat entries are scattered through every country; no leading column to descend on
<code>country = 'MA' AND city > 'M'</code>	yes, partially	descend on country, then range on city

Think of the phone book: you can find everyone named Alaoui, and every Alaoui named Sara, but you cannot find everyone named Sara without reading the whole book. **Column order matters.** Put first the column you filter on most often and most selectively. Lab 3, Part 3 has you verify all three cases with `EXPLAIN`.

Summary

Hash: $O(1)$ equality, no ranges. Bitmap: one bitmap per value for low-cardinality columns, fast AND/OR, bad for updates; PostgreSQL builds them on the fly. Partial: index a subset of rows. Expression: index $f(\text{column})$. Covering: put every needed column in the index so the heap is never read. Multi-column: sorted by the first column then the second; only predicates on a leftmost prefix can use it.

5 Choosing Indexes

5.1 When an index helps, and when it does not

Trade-offs: Helps versus hurts

Helps

Lookups returning few rows.

Joins on the indexed key (Chapter 4: index nested-loop join).

`ORDER BY` or a range on the key (walk the leaves, no sort).

Every index speeds some reads and slows *every* write. Index for the queries you actually run.

Hurts or is wasted

Queries returning most of the table (scan wins).

Tiny tables: a few pages, already cheap to scan.

Write-heavy columns: every write pays the maintenance.

5.2 The cost of having an index

- **Space.** A B+ tree on an 8-byte key is roughly 16 bytes per row plus overhead; on a million-row table that is 20–30 MB, and on a wide key such as an email address it can approach the table's own size. Five indexes can double the disk footprint.
- **Write amplification.** Every `INSERT` writes the heap page *and* one leaf page per index (plus occasional splits). Every `UPDATE` that changes an indexed column removes and reinserts the entry. Every `DELETE` leaves dead entries to clean up. A table with six indexes does about seven page writes per inserted row instead of one. Lab 3, Part 6 measures this directly.
- **Optimiser effort.** More indexes mean more candidate plans to cost, and more chances for a wrong estimate to pick a bad one.

Technique: Design discipline for indexes

1. Start from the workload: list the important queries and what they filter, join and sort on.
2. For each, ask: how many rows does it return? Few: index candidate. Most: no index will help.
3. Prefer one multi-column index that serves several queries over several single-column ones, ordering columns by the leftmost-prefix rule.

4. Consider partial or covering variants when the query pattern is narrow and fixed.
5. Create it, run `EXPLAIN ANALYZE` before and after, and keep it only if the plan changed and got faster.
6. Do not index “just in case”. Unused indexes cost writes and space for no benefit.

Worked example: Pause and check: why not index every column?

If indexes make lookups thousands of times faster, why not create one on every column of every table?

Because each index has ongoing costs. Every write must update every relevant index, so inserts and updates slow down in proportion to the number of indexes. Indexes consume significant disk space. Most “just in case” indexes are never chosen by the optimiser for any query, so they pay full cost for no benefit. And too many choices can lead the optimiser astray. Index the columns your actual queries filter, join or sort on, then measure.

6 Indexing in PostgreSQL

6.1 Creating indexes

```
-- default B-tree
CREATE INDEX idx_cust_country ON customers(country);

-- multi-column (leftmost-prefix rule applies)
CREATE INDEX idx_orders_cust_date ON orders(cust_id, order_date);

-- expression index
CREATE INDEX idx_lower_name ON customers(lower(name));

-- partial index
CREATE INDEX idx_ma ON customers(id) WHERE country = 'MA';

-- covering index (index-only scans for queries needing only these
  columns)
CREATE INDEX idx_orders_cust_amt ON orders(cust_id) INCLUDE (amount);

-- other index types
CREATE INDEX idx_email_hash ON customers USING hash (email);

-- see whether the planner uses it
EXPLAIN ANALYZE SELECT * FROM customers WHERE country = 'MA';
```

Index types available: `btree` (the default, a B+ tree), `hash`, `gin` (for arrays, JSON and full-text search), `gist` (for geometric and range data), `brin` (block-range summaries for huge, naturally ordered tables), `spgist`. Also useful:

```

\d orders -- lists the table's
          indexes
SELECT pg_size_pretty(pg_relation_size('idx_orders_date')); --
          index size
SELECT indexrelname, idx_scan FROM pg_stat_user_indexes;      -- how
          often each index is used

```

The last query is how you find “just in case” indexes: `idx_scan = 0` after a week of production traffic means nobody uses it.

6.2 Reading the plan

Interface / API: The four scan types you will see

- **Seq Scan:** read the heap start to finish. Means no useful index exists, or the query returns too much of the table for an index to pay off.
- **Index Scan:** descend the index, then fetch each matching row from the heap, one random page read per row. Best for few matches.
- **Index Only Scan:** the index alone answered the query (a covering index, and the visibility map confirmed the rows are visible). No heap reads at all.
- **Bitmap Index Scan + Bitmap Heap Scan:** collect all matching row ids from the index into a bitmap, sort them by page, then read each heap page once in physical order. PostgreSQL’s choice for a medium number of matches, where a plain Index Scan would re-read pages and a Seq Scan would read too many.

```

Index Scan using idx_cust_name on customers (cost=0.42..8.44 rows=1 width=40)
                                         (actual time=0.031..0.033 rows=1 loops
=1)
Index Cond: (name = 'Customer 7777')
Buffers: shared hit=4

```

Read Buffers: `shared hit=4`: four pages touched, three for the tree and one for the heap row, exactly the $h + 1$ from Section 3. Compare the same query without the index:

```

Seq Scan on customers (cost=0.00..234.00 rows=1 width=40)
                    (actual time=1.9..4.7 rows=1 loops=1)
Filter: (name = 'Customer 7777')
Rows Removed by Filter: 9999
Buffers: shared hit=109

```

109 pages, every one of them, and 9,999 rows read and thrown away. That ratio, 109 to 4, is what Lab 3 asks you to report as the speedup.

Toggling `SET enable_seqscan = off;` forces the planner to prefer an index even when it thinks a scan is cheaper. It is a teaching trick for comparing plans side by side, not something to do in production, because the planner was usually right.

7 Lab 3 Companion: Measuring the Payoff and the Cost

Lab 3 turns every claim in this chapter into a number. Here is what each part is testing and what you should expect to see.

Technique: Part by part

1. **Baseline.** Both queries show **Seq Scan**; note the **Buffers** count, which is roughly B for each table. This is the Chapter 2 cost, measured.
2. **Add indexes.** The point lookup on **name** flips to **Index Scan** with a handful of buffers. The ten-day range on **order_date** probably becomes a **Bitmap Heap Scan**: about 3% of the year, enough matches that PostgreSQL prefers to sort the row ids by page before reading the heap.
3. **Prefix rule.** With **idx_oc** on (**cust_id**, **order_date**): the query on **cust_id** alone uses it; the query on both uses it; the query on **order_date** alone does *not* (it may use **idx_orders_date** from Part 2 instead, so drop that first if you want to see the Seq Scan).
4. **Selectivity flip.** Amounts are uniform on 5–105, so **amount** > 104 matches about 1%, > 60 about 45%, > 20 about 85%. Expect Index or Bitmap scan at 1%, and Seq Scan at 45% and 85%. Narrow the threshold between until the plan changes; the fraction where it flips is your answer, typically somewhere between 5% and 20% depending on the table's physical order and the **random_page_cost** setting.
5. **Expression index.** **lower(name) = ...** with only **idx_cust_name** present: Seq Scan, because the tree is sorted by **name**, not **lower(name)**. After creating **idx_lower_name**: Index Scan.
6. **Write cost.** Inserting 100,000 rows with three indexes on **orders** versus none. Expect a slowdown of roughly 2–4×; each row now writes one heap page plus three leaf pages, and the random leaf positions defeat sequential writing.

Debugging: If the numbers do not behave

- **Still Seq Scan after creating an index.** Run **ANALYZE**; so the planner has fresh statistics. Then check how many rows the query returns: if it is a large fraction, the Seq Scan is the right answer and the index is not at fault.
- **Timings jump around.** The first run reads from disk, the second from the buffer pool. Run each query twice, report the second (warm) time, and say so. Better: compare **Buffers** counts, which do not depend on caching.
- **The range query uses Bitmap Heap Scan, not Index Scan.** That is expected and is still the index being used. Look for **Bitmap Index Scan on idx_orders_date** beneath it.
- **Index Only Scan did not appear for a covering index.** The visibility map must be up to date; run **VACUUM orders**; after bulk

inserts.

8 Tricks, Techniques and Classic Problems

Warning / Gotcha: Common student mistakes

- **“I created an index, so the query will use it.”** Only if the query returns few enough rows. Check with `EXPLAIN`; a Seq Scan on a query returning half the table is correct behaviour.
- **Forgetting the heap fetch.** A B+ tree lookup is h reads to the leaf *plus one* for the row. An index scan returning k scattered rows costs about $h + k$, not h .
- **Expecting a hash index to help a range or an ORDER BY.** It cannot; hashing destroys order.
- **Putting the wrong column first in a multi-column index.** `(city, country)` does not serve `WHERE country = 'MA'`.
- **Indexing name and querying lower(name).** Different sort key; use an expression index.
- **“More indexes are always safer.”** Each one slows every write and most are never used.
- **Confusing height with $\log_2$.** A B+ tree’s height is $\log_f N$ with fanout f in the hundreds, so about 3–4, not 20.

Worked example: Classic exam and interview problems

P1. A B+ tree with fanout about 100 indexes a million-row table whose heap is 12,821 pages. Height? Reads for one lookup by key? Compare with a full scan.

Height $\lceil \log_{100} 10^6 \rceil = 3$. About 3 reads down the tree plus 1 for the heap row, so 4 page reads. A full scan is 12,821 reads; the index is roughly $3,000\times$ cheaper for this lookup.

P2. Same table. Cost of a range that matches 500 rows spread over 7 leaves?

3 (descend) + 7 (leaves) + 500 (heap fetches, worst case one page each) ≈ 510 reads. Still 25 times cheaper than the scan. At 50,000 matching rows it would be about 50,000, four times *worse* than the scan, and the planner would choose the scan.

P3. Which index type for each? (a) `WHERE email = 'a@x.com'`, unique. (b) `WHERE order_date BETWEEN ... AND ....` (c) `WHERE country = 'MA'` on an analytics table with 5 countries. (d) `WHERE lower(name) = 'sara'`. (e) frequent `WHERE status = 'active'` where most rows are inactive.

(a) B-tree (or hash): exact match. (b) B-tree on `order_date`: ranges need order. (c) Bitmap (bitmap heap scan): low cardinality. (d) Expression index on `lower(name)`. (e) Partial index `WHERE status = 'active'`: small and targeted.

P4. Why does a B+ tree keep all data in the leaves and link them?

Internal nodes then hold only routing keys, maximising fanout and minimising height; and because every search ends at the leaf level, a range query can descend once and walk the chain instead of re-searching per key.

P5. Leaves hold 4 entries. Insert 35 into a leaf (30, 40, 50, 55) whose parent is (30 | 60).

The leaf is full: split into (30, 35) and (40, 50, 55), push 40 up. Parent becomes (30 | 40 | 60). (Whether 40 stays in the new leaf as well as going up is a convention; in a B+ tree it does, since all keys must appear in leaves.)

P6. An index on (a, b, c). Which predicates can use it? a = 1; a = 1 AND b = 2; b = 2; a = 1 AND c = 3; a > 1 AND b = 2.

Yes; yes; no (no leading column); partially (descend on a, then filter c inside the a = 1 run, since b is skipped); partially (range on a, then b is only a filter, because within a range of a the b values are not contiguous).

P7. Why is a secondary index scan that returns 40% of the table slower than a sequential scan that reads 100% of it?

The index returns row ids in key order, which is unrelated to heap order, so each match is a random page read, and pages are re-read many times. The scan reads each page exactly once, sequentially. Random reads are far more expensive per page than sequential ones, and 40% of a million rows is far more reads than B pages.

P8. Name one column you would index in a typical orders table and one you would not, with a reason.

Index `cust_id`: it is the join key to customers and queries usually ask for one customer's orders, returning few rows. Do not index `amount` on its own if queries only ever aggregate over all orders: they return every row, so a scan is chosen anyway and the index only slows inserts.

Summary: Chapter 3 in seven lines

1. An index maps key values to row ids, turning $O(B)$ scans into $O(\log B)$ or $O(1)$ lookups. It lives beside the heap.
2. The B+ tree: routing keys inside, all entries in chained sorted leaves, one page per node, always balanced. Height about 3–4.
3. Point lookup $h + 1$ reads; range: descend once, walk the leaves; insert: split a full leaf and push a key up.
4. Hash for equality only; bitmap for low-cardinality analytics; partial, expression and covering to fit a known query; multi-column obeys the leftmost-prefix rule.
5. Selectivity decides everything: an index scan costs about one random read per match, so past a few percent of the table the sequential scan wins and the planner picks it.
6. Every index slows every write and takes space. Index for the queries you run, then measure with `EXPLAIN (ANALYZE, BUFFERS)`.
7. Next: Chapter 4, how scans, sorts and joins actually execute.